



Department of Electronics And Communication

Course Level Problem Based Learning

Course Title: DDHDL Verilog

Course Code:22ECE42

Semester: Fourth Semester

Project Report Titled

FPGA implementation of Adaptive FIR filter using LMS algorithm

Submitted By

Sl No.	USN	Name
1	4SO24EC024	Chaman R Shetty
2	4SO24EC026	Chirag Mundodi
3	4SO24EC027	Chitthara K S
4	4SO24EC044	Hithashree

Under the Guidance of

P.C Vijay Ganesh

(Assistant Professor , B.E , M. Tech, MIEEE)

PBL SECONDARY RESEARCH REPORT

Chapter 1: INTRODUCTION

1.1 Background:

Digital signal processing is widely used in applications such as communication systems, audio processing, and environmental monitoring. However, real-world signals are often affected by noise, which reduces system accuracy and performance. To address this issue, filtering techniques are used to extract useful information from noisy signals.

Finite Impulse Response (FIR) filters are commonly used due to their stability and linear phase characteristics. In dynamic environments where signal conditions change over time, adaptive filtering becomes necessary. Adaptive FIR filters, combined with the Least Mean Squares (LMS) algorithm, can automatically update their coefficients to minimize error and improve signal quality.

Field Programmable Gate Arrays (FPGAs) provide an efficient platform for implementing such filters due to their high speed, parallel processing capability, and reconfigurability. By using optimized hardware components such as shift registers, fast adders, and pipelining techniques, high-performance real-time signal processing can be achieved.

1.2 Problem Statement:

Real-time biomedical and environmental monitoring systems require efficient digital signal processing. Software-based FIR filtering can introduce latency and increased power consumption in wearable and IoT devices. Design and implement an FPGA-based hardware system capable of performing real-time digital filtering for streaming input data.

1.3 Objectives:

The design focuses on developing an adaptive FIR filter architecture based on the LMS algorithm, optimized for efficient FPGA implementation. The filter employs a shift register–based delay line to store input samples, reducing resource utilization and improving hardware efficiency. A 16-bit Carry Lookahead Adder (CLA) is incorporated to achieve high-speed arithmetic operations, significantly enhancing performance compared to conventional ripple carry adders. Optimized multiplier units are integrated to perform fast and accurate coefficient multiplications with input samples. To further improve throughput, a pipelined architecture is implemented, enabling parallel processing and high-speed operation. The overall design is optimized to maintain a balance between throughput, latency, area utilization, and power consumption. Additionally, the system is designed to support real-time signal processing with minimal delay. Finally, the architecture is verified using HDL simulation tools, and its performance is evaluated in terms of speed, area, and computational accuracy.

Chapter 2: Literature Review (Secondary Research)

2.1 Review of Existing Work:

Paper 1: FPGA-Based Adaptive FIR Filter Using LMS Algorithm (Heba H. Ali et al., 2025)

Architecture Used: FPGA-based Adaptive FIR Filter using LMS algorithm (Multiplier-based, BCD multiplier optimized design)

Throughput Achieved: Very high speed (~1282 MHz operating frequency)

Resource Utilization: ~151 logic elements, 12 registers (BCD multiplier implementation)

Limitations:

Limited to small filter order (3-tap design)

Focus mainly on multiplier optimization rather than overall system architecture

No detailed power analysis

Paper 2: Low Power and Area Efficient FIR Filter Using Adaptive LMS Algorithm (M. Dasharatha et al., 2017)

Architecture Used: LMS-based Adaptive FIR Filter (MAC-based architecture)

Throughput Achieved: Moderate (not explicitly quantified, but optimized for efficiency)

Resource Utilization: Significant reduction (~46% area improvement, 23% power reduction)

Limitations:

Trade-off between convergence speed and accuracy

Fixed-point implementation limits precision

Performance depends heavily on step size parameter

Paper 3: Multiplier-Less LMS Adaptive Filter using Distributed Arithmetic (Shawez Ahmad et al., 2021)

Architecture Used: Distributed Arithmetic (DA) based LMS Adaptive Filter (Multiplier-less architecture)

Throughput Achieved: Improved throughput (~18% improvement)

Resource Utilization: Reduced area (~5% improvement), reduced power (~3.5%)

Limitations:

Increased latency (up to 4× higher)

Memory size grows exponentially with filter order

Complexity in LUT design and memory management

Paper 4: Adaptive FIR Filter Based on Distributed Arithmetic and LMS Algorithm (IJERT, 2014)

Architecture Used: Distributed Arithmetic + LMS Hybrid Architecture

Throughput Achieved: Moderate to high (optimized for low power systems)

Resource Utilization: Low area and low power consumption (optimized FPGA implementation)

Limitations:

Increased design complexity

Latency due to DA implementation

Limited scalability for higher-order filters

2.2 Comparative Analysis Table

Paper	Architecture	Throughput	Area	Limitation
1	FPGA LMS (BCD Multiplier)	~1282 MHz	151 Logic Elements, 12 Registers	Limited to small tap size, no power analysis
2	LMS MAC- based	Moderate	Reduced (~46% area)	Precision issues, convergence dependency
3	DA-based LMS	Improved (~18%)	Reduced (~5%)	High latency, memory growth
4	DA + LMS Hybrid	Moderate- High	Low power & area	Complex design, latency issues

2.3 Research Gap Identification

Trade-off Between Speed and Area: Paper 1 focuses mainly on achieving high-speed performance using multiplier-based structures, but it does not consider optimization of power and scalability, leading to higher resource usage. In contrast, Papers 3 and 4 aim to reduce hardware area through techniques like distributed arithmetic and resource sharing, but this results in increased latency, affecting overall speed.

Latency vs Resource Optimization Issue: Multiplier-less (DA) designs help in reducing hardware complexity and FPGA resource utilization, but they introduce higher latency due to sequential processing. On the other hand, multiplier-based designs provide

faster computation and lower delay, but they consume more logic elements and DSP resources.

Scalability Limitations: Most of the existing designs are limited to low-order filters such as 3-tap structures. As the number of taps increases, memory requirements and computational complexity grow rapidly, making the design less efficient and harder to scale for higher-order filters.

Lack of Balanced Architecture: Existing works tend to optimize only one aspect of the design. Paper 1 focuses on speed, whereas Papers 3 and 4 focus on area and power reduction. There is a lack of architectures that provide a balanced trade-off between speed, area, and power.

Limited Real-Time Adaptability: Some designs do not efficiently support real-time coefficient updates required in adaptive filtering. In certain cases, the adaptation process introduces additional delay, reducing the effectiveness of the system in real-time applications.

Conclusion (Research Gap Statement)

Most existing designs focus on optimizing either speed (multiplier-based FPGA designs) or area/power (DA-based designs), but not both simultaneously.

Additionally, latency and scalability remain major challenges. Therefore, a balanced FPGA-based adaptive FIR filter architecture is required that achieves high throughput, low area utilization, reduced latency, and efficient real-time performance, making it suitable for embedded and real-time applications.

Chapter 3: Proposed System Design

Why this architecture was selected

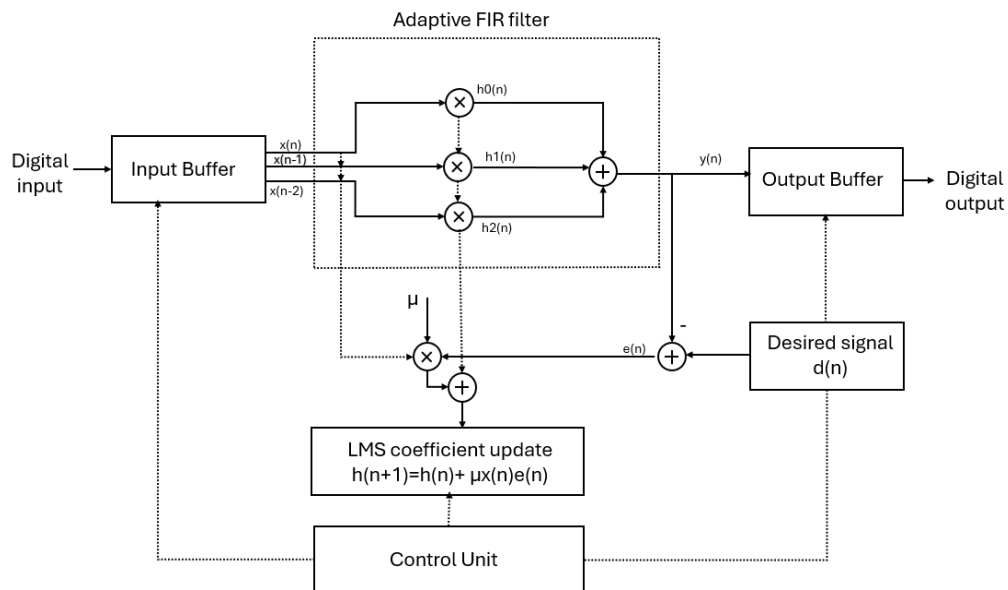
An FPGA-based 10-tap LMS adaptive FIR filter with 16-bit input precision is selected to achieve accurate, high-speed, and real-time signal processing. The choice of a 10-tap FIR filter provides a balanced trade-off between filtering performance and hardware complexity, offering improved noise reduction and signal approximation compared to lower-order filters while keeping resource usage manageable. The LMS algorithm is adopted due to its low computational complexity and ease of implementation, enabling continuous adaptation of filter coefficients to minimize error. A 16-bit data path is used to enhance numerical accuracy and dynamic range. To improve speed, a Carry Look Ahead (CLA) adder is implemented to reduce propagation delay, while a pipelined multiplier architecture increases throughput by allowing parallel processing of operations. The delay line is realized using shift registers with D flip-flops, ensuring efficient and synchronized data handling for the tap inputs. Compared to conventional designs, this architecture provides higher speed, better precision, improved throughput, and scalability, making it well-suited for real-time adaptive filtering applications on FPGA platforms.

How it improves over existing literature

The proposed adaptive FIR filter design achieves improved filtering performance by using a 10-tap structure, which provides better noise suppression and signal approximation compared to lower-order filters such as 3-tap systems. High-speed operation is achieved through the use of Carry Lookahead Adders (CLA), which reduce computational delay, along with pipelined multipliers that minimize the critical path. This pipelined architecture also increases throughput by enabling multiple operations to occur simultaneously, thereby enhancing overall system efficiency. The use of a 16-bit data path improves precision and accuracy compared to lower bit-width implementations. Additionally, the design ensures efficient hardware utilization through D flip-flop-based shift registers, which offer a structured and resource-optimized method for implementing delay elements. The architecture is

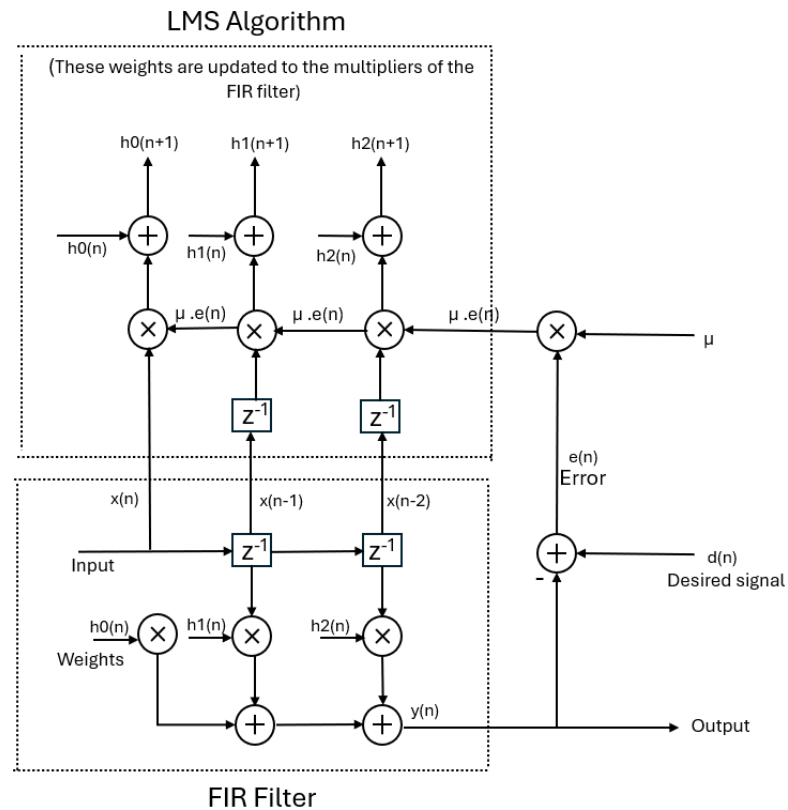
scalable, allowing easy extension to higher-order filters with minimal modifications. Furthermore, real-time adaptability is achieved using the LMS algorithm, which continuously updates filter coefficients to minimize error and maintain optimal filtering performance.

3.2 Block Diagram



3.3 Architecture Diagram

For 3 Tap

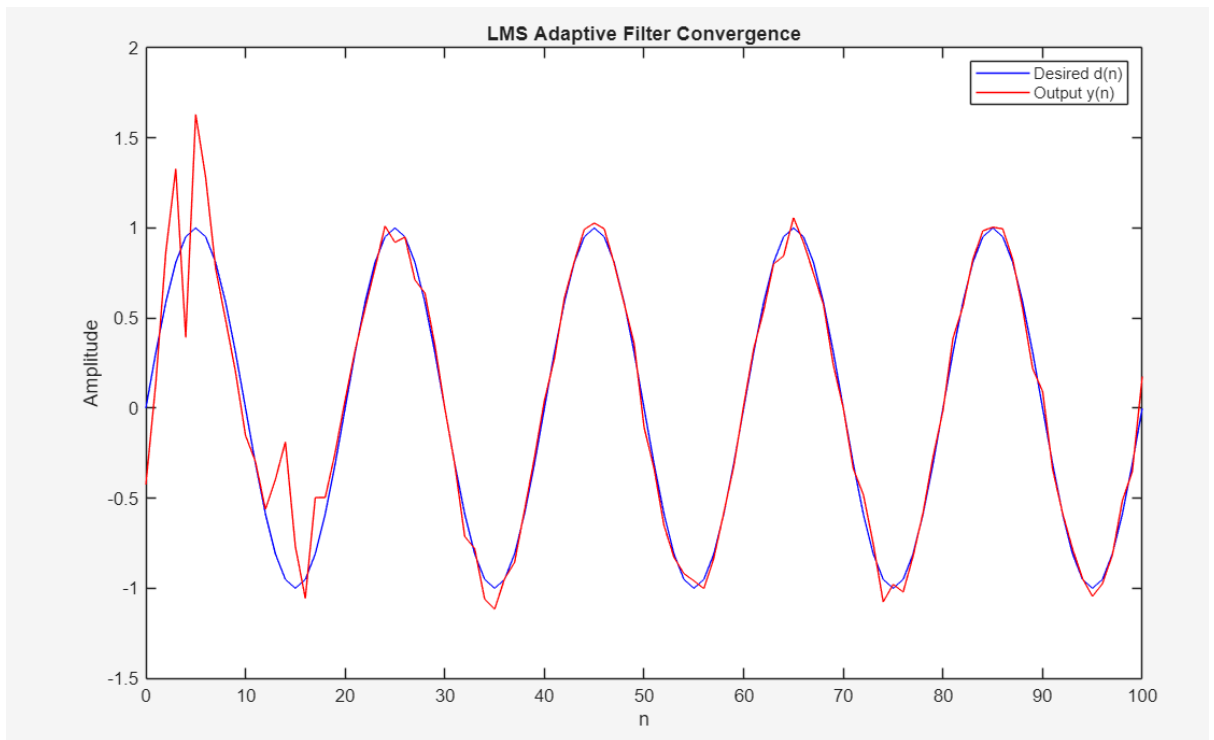


Graph of $d(n)$ (desired signal) and $y(n)$ (output signal)

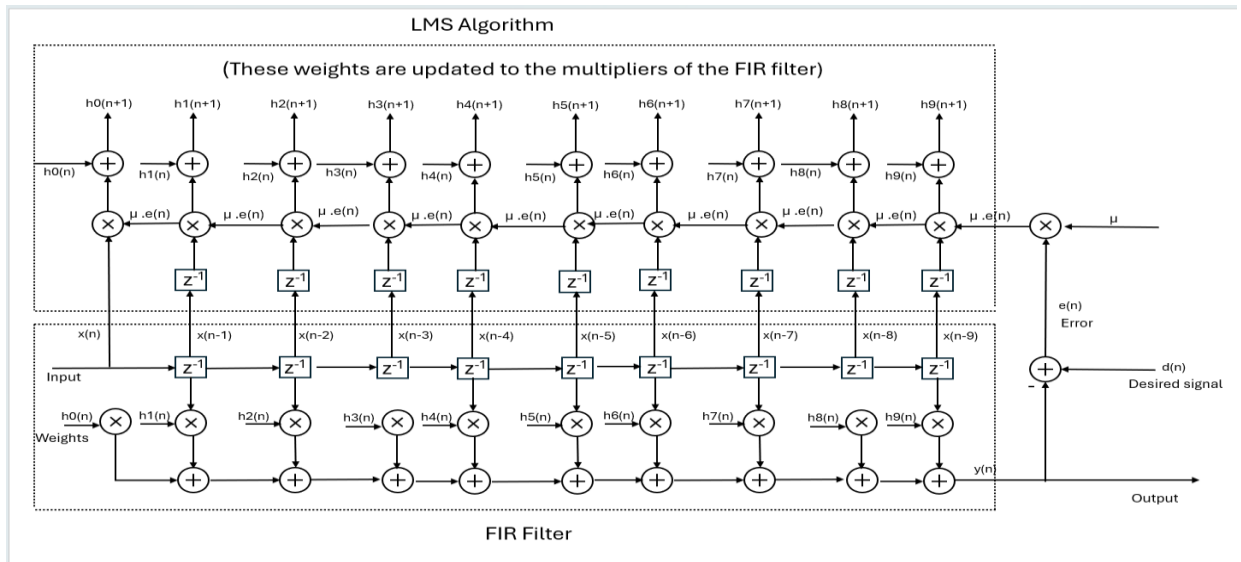
```
n = 0:100;
d = sin(0.1*pi*n);           % desired signal
y = d + 0.5*randn(size(n)); % initial noisy output

% simulate convergence
for i = 20:100
    y(i) = d(i) + 0.05*randn;
end

plot(n, d, 'b', n, y, 'r')
legend('Desired d(n)', 'Output y(n)')
xlabel('n')
ylabel('Amplitude')
title('LMS Adaptive Filter Convergence')
```



For 10 Taps:



1. Output of FIR filter

For an FIR filter :

$$y(n) = \sum_{k=0}^{N-1} h(n) x(n - k)$$

Where:

- $y(n)$ = output signal
- $x(n-k)$ = input signal (delayed samples)
- $h(n)$ = adaptive filter coefficients (time-varying)
- N = number of taps

2. Error signal

$$e(n) = d(n) - y(n)$$

- $d(n)$ = desired signal
- $e(n)$ = error used for adaptation

3. LMS Formula

$$h(n+1) = h(n) + \mu x(n) e(n)$$

- μ = step size
- $h(n+1)$ = updated filter coefficient

Chapter 4: Leaf Cell Design

4.1 Identification of Leaf Cells

Adder/Subtractor : 16-bit Carry Look Ahead Adder (CLA)

Multiplier : Pipelined Multiplier

Delay Element : Shift Register

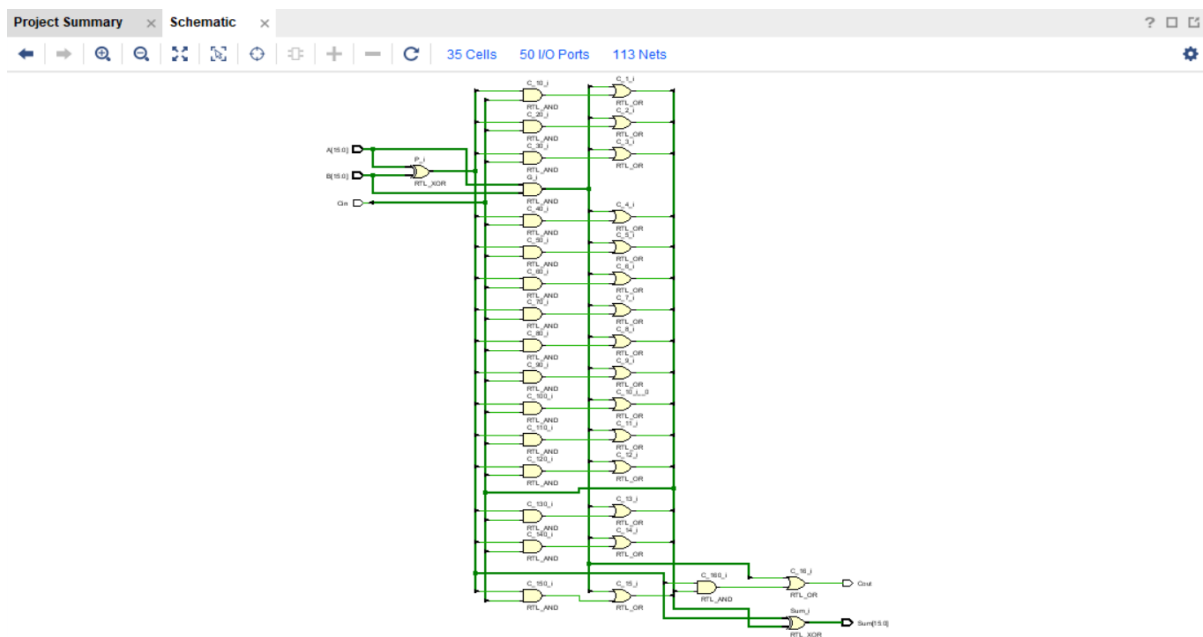
1. Carry Look Ahead Adder (CLA)

Functionality:

The 16-bit Carry Look-Ahead Adder (CLA) performs fast addition and subtraction of two 16-bit binary numbers. It improves speed by calculating carry signals in advance using generate and propagate logic instead of waiting for carry to ripple through each bit. This makes it suitable for high-speed applications like ECG signal processing where quick arithmetic operations are required.

RTL Design :

In RTL design, the 16-bit CLA is implemented using a hierarchical structure by combining smaller blocks (typically 4-bit CLA units). Each block computes its own propagate and generate signals, and higher-level logic determines the carry inputs for each block. The design is purely combinational and can also support subtraction using 2's complement by inverting one input and adding a carry-in.

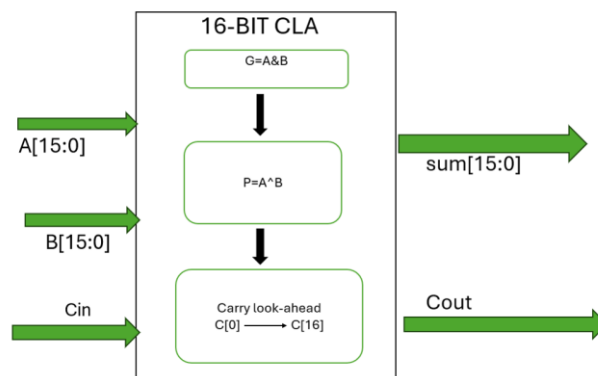


Optimization used:

The main optimization in CLA is the reduction of carry propagation delay by computing carries in parallel. This significantly increases speed compared to ripple carry adders. Although it uses more hardware due to additional logic for carry

generation, the trade-off is acceptable in FPGA implementations where timing performance is critical.

Diagram:



Main code of carry look ahead adder:

```

23 module clal6 (
24     input  [15:0] A,
25     input  [15:0] B,
26     input  Cin,
27     output [15:0] Sum,
28     output Cout
29 );
30
31 wire [15:0] G, P;
32 wire [16:0] C;
33
34 assign C[0] = Cin;
35
36 // Generate & Propagate
37 assign G = A & B;
38 assign P = A ^ B;
39
40 // Carry Look-Ahead Logic
41 assign C[1] = G[0] | (P[0] & C[0]);
42 assign C[2] = G[1] | (P[1] & C[1]);
43 assign C[3] = G[2] | (P[2] & C[2]);

```

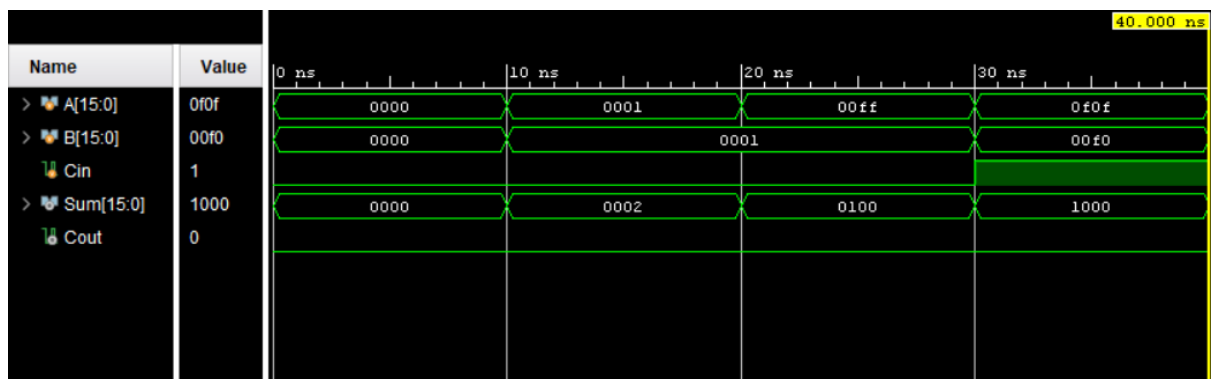
Testbench code:

```

25 module clal6_tb;
26
27     // Inputs
28     reg [15:0] A;
29     reg [15:0] B;
30     reg Cin;
31
32     // Outputs
33     wire [15:0] Sum;
34     wire Cout;
35
36     // Instantiate DUT
37     clal6 uut (
38         .A(A),
39         .B(B),
40         .Cin(Cin),
41         .Sum(Sum),
42         .Cout(Cout)
43     );
44
45     initial begin
46         // Display output
47         $monitor("Time=%0t | A=%h B=%h Cin=%b | Sum=%h Cout=%b",
48             $time, A, B, Cin, Sum, Cout);
49
50         // Test cases
51         A = 16'h0000; B = 16'h0000; Cin = 0; #10;
52         A = 16'h0001; B = 16'h0001; Cin = 0; #10;
53         A = 16'h00FF; B = 16'h0001; Cin = 0; #10;
54         A = 16'h0F0F; B = 16'h00F0; Cin = 1; #10;
55
56         $finish;
57     end
58
59 endmodule

```

Simulation Waveform:



AND gate:

a=0; b=0;

#10;

a=0; b=1;

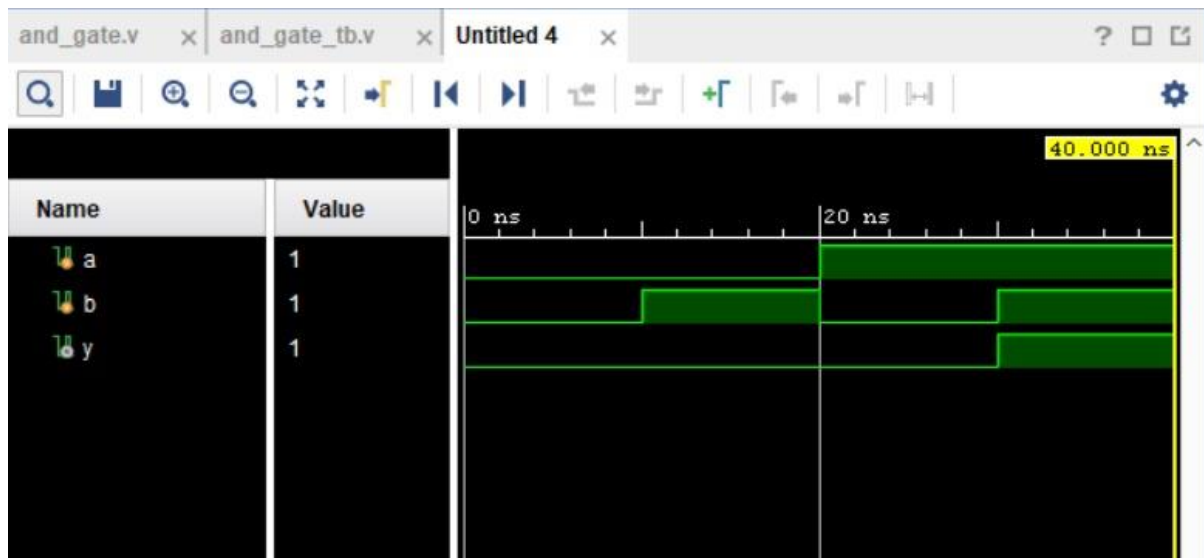
#10;

a=1; b=0;

#10;

a=1; b=1

Waveform of AND gate:



XOR gate:

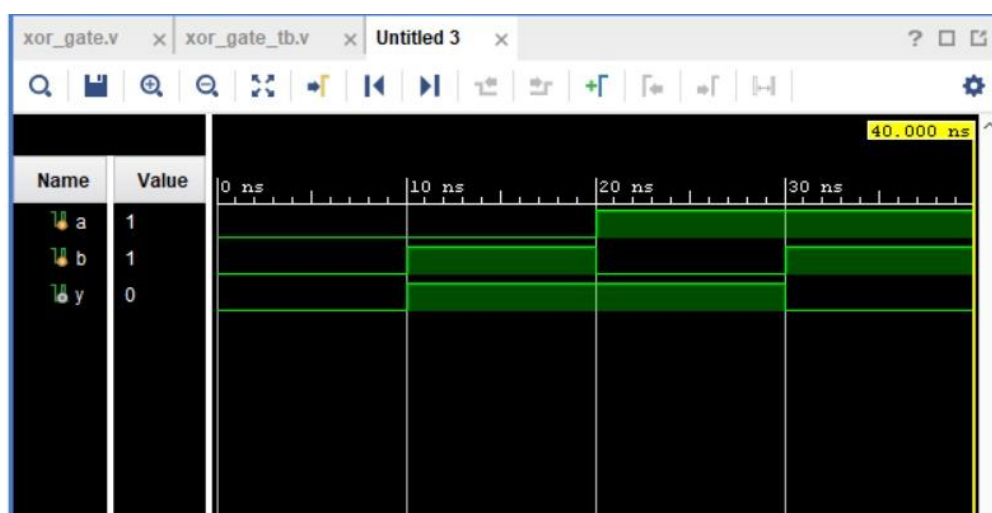
a=0; b=0;

#10 a=0; b=1;

#10 a=1; b=0;

#10 a=1; b=1;

Waveform:



2. Pipelined Multiplier:

Functionality:

Main code for pipelined multiplier:


```

23 module mult16_pipelined (
24     input clk,
25     input rst,
26     input [15:0] A,
27     input [15:0] B,
28     output reg [31:0] P
29 );
30
31     reg [15:0] A_reg, B_reg;
32     reg [31:0] mult_reg;
33
34     // Stage 1: Register inputs
35     always @(posedge clk) begin
36         if (rst) begin
37             A_reg <= 0;
38             B_reg <= 0;
39         end else begin
40             A_reg <= A;
41             B_reg <= B;
42         end
43     end
44
45     // Stage 2: Multiply
46     always @(posedge clk) begin
47         mult_reg <= A_reg * B_reg;
48     end
49
50     // Stage 3: Output register
51     always @(posedge clk) begin
52         P <= mult_reg;
53     end
54
55 endmodule

```

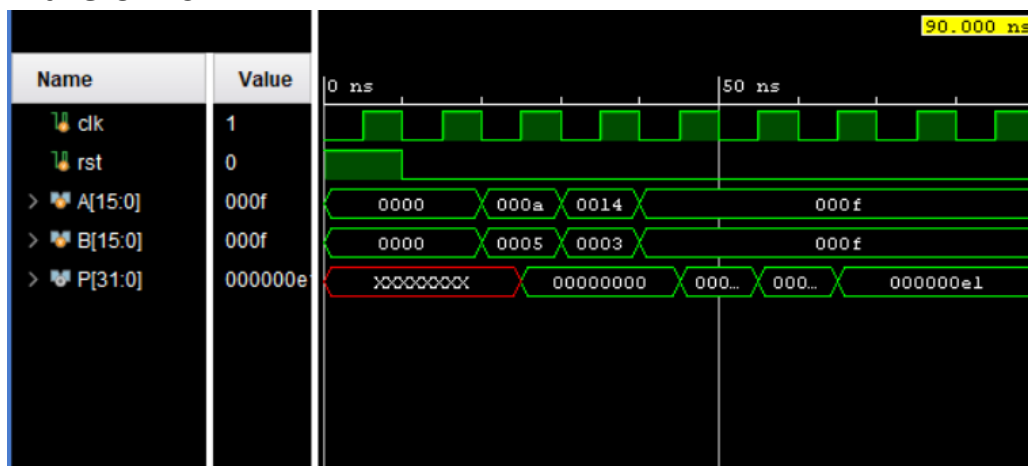
Testbench code:

```

23 module mult16_pipelined (
24     input clk,
25     input rst,
26     input [15:0] A,
27     input [15:0] B,
28     output reg [31:0] P
29 );
30
31     reg [15:0] A_reg, B_reg;
32     reg [31:0] mult_reg;
33
34     // Stage 1: Register inputs
35     always @(posedge clk) begin
36         if (rst) begin
37             A_reg <= 0;
38             B_reg <= 0;
39         end else begin
40             A_reg <= A;
41             B_reg <= B;
42         end
43     end
44
45     // Stage 2: Multiply
46     always @(posedge clk) begin
47         mult_reg <= A_reg * B_reg;
48     end
49
50     // Stage 3: Output register
51     always @(posedge clk) begin
52         P <= mult_reg;
53     end
54
55 endmodule

```

Waveforms:



Half adder:

A=0; B=0;

#10;

A=0; B=1;

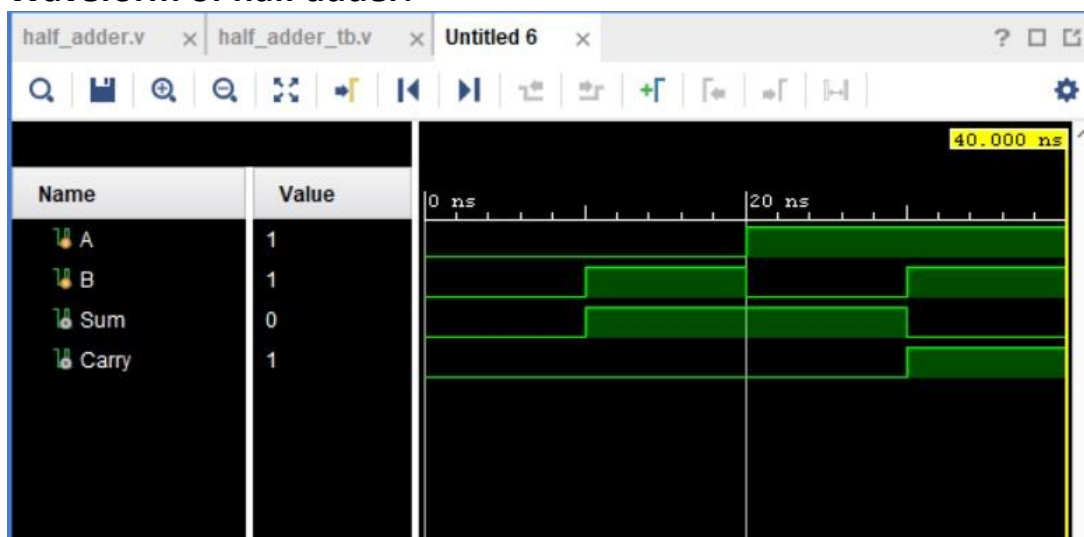
#10;

A=1; B=0;

#10;

A=1; B=1

Waveform of half adder:



Full adder:

A=0; B=0; Cin=0; #10;

A=0; B=0; Cin=1; #10;

A=0; B=1; Cin=0; #10;

A=0; B=1; Cin=1; #10;

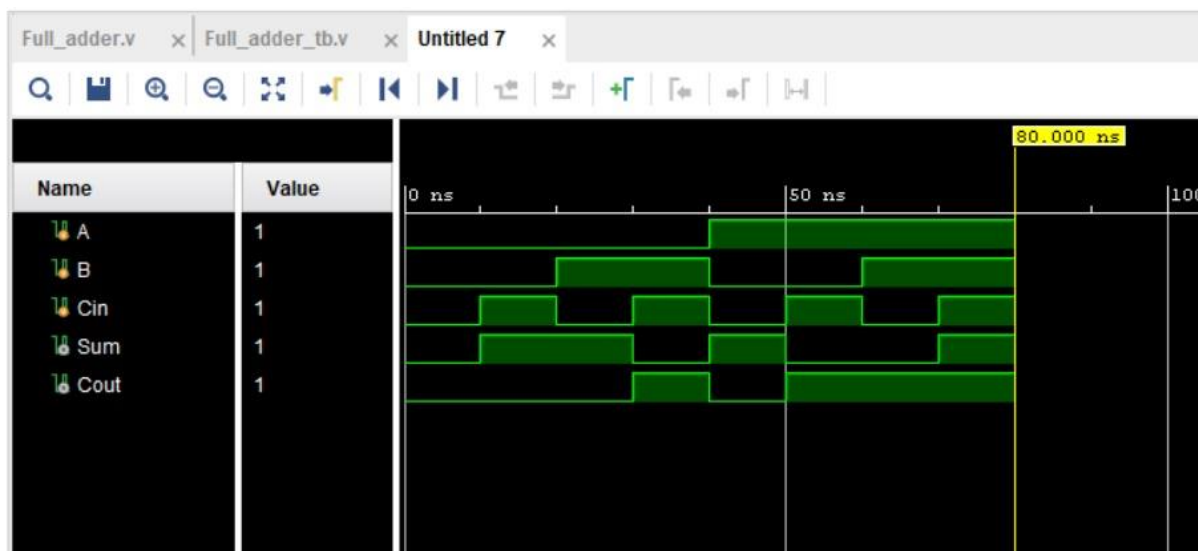
A=1; B=0; Cin=0; #10;

A=1; B=0; Cin=1; #10;

A=1; B=1; Cin=0; #10;

A=1; B=1; Cin=1; #10;

Waveform of full adder:



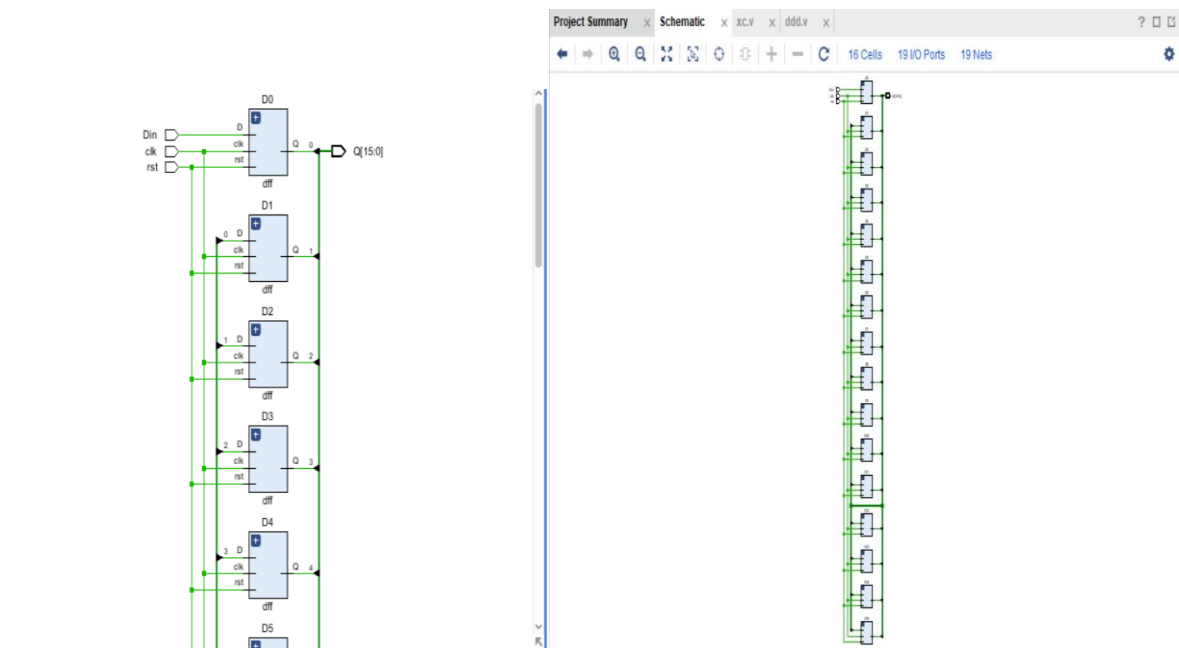
3. Shift Register (Delay Element)

Functionality:

The leaf cell acts as a delay element in an FIR filter by storing the current and previous input samples such as $x(n)$, $x(n-1)$, $x(n-2)$, and so on. This forms a delay line that aligns input samples in time, enabling proper convolution with filter coefficients during FIR processing and supporting continuous data streaming.

RTL Design:

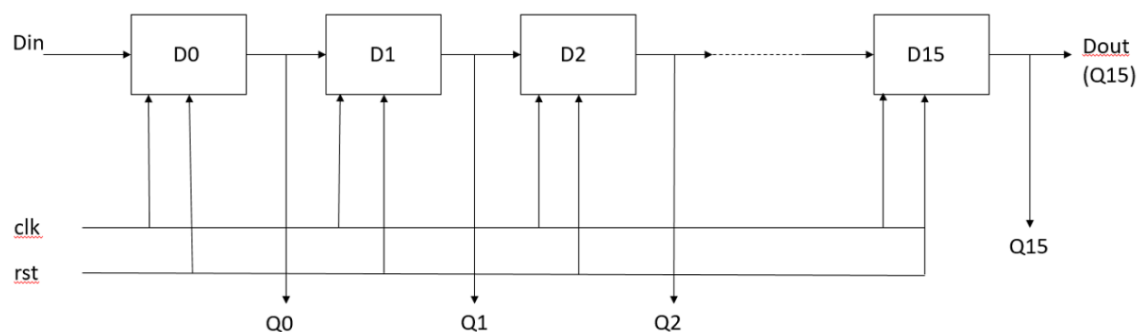
At the RTL level, the leaf cell is implemented as a series of D flip-flops connected in sequence. On every clock cycle, the input sample is loaded into the first register, and existing data shifts to the next stage. This creates a pipelined structure where each register holds a delayed version of the signal, forming the required delay chain with simple sequential logic.



Optimization used:

The design is optimized for simplicity and efficiency by using only basic hardware elements (flip-flops), resulting in low resource utilization. It maps efficiently onto FPGA architectures by leveraging register resources and avoids complex combinational logic. Additionally, the pipelined structure enables real-time streaming and high-throughput operation with minimal latency.

Diagrams:



Main Code for Shift register:

```

23 module shift_register_16 (Q, Din, clk, rst);
24
25     output [15:0] Q;
26     input Din, clk, rst;
27
28     wire [14:0] w;
29
30     dff D0 (Q[0], Din, clk, rst);
31
32
33     dff D1 (Q[1], Q[0], clk, rst);
34     dff D2 (Q[2], Q[1], clk, rst);
35     dff D3 (Q[3], Q[2], clk, rst);
36     dff D4 (Q[4], Q[3], clk, rst);
37     dff D5 (Q[5], Q[4], clk, rst);
38     dff D6 (Q[6], Q[5], clk, rst);
39     dff D7 (Q[7], Q[6], clk, rst);
40     dff D8 (Q[8], Q[7], clk, rst);
41     dff D9 (Q[9], Q[8], clk, rst);
42     dff D10 (Q[10], Q[9], clk, rst);
43     dff D11 (Q[11], Q[10], clk, rst);
44     dff D12 (Q[12], Q[11], clk, rst);
45     dff D13 (Q[13], Q[12], clk, rst);
46     dff D14 (Q[14], Q[13], clk, rst);
47     dff D15 (Q[15], Q[14], clk, rst);
48
49 endmodule
50
51
52 module dff (Q, D, clk, rst);
53
54     output reg Q;
55     input D, clk, rst;
56
57     always @(posedge clk or posedge rst)
58     begin
59         if (rst)
60             Q <= 1'b0;
61         else
62             Q <= D;
63     end
64
65 endmodule

```

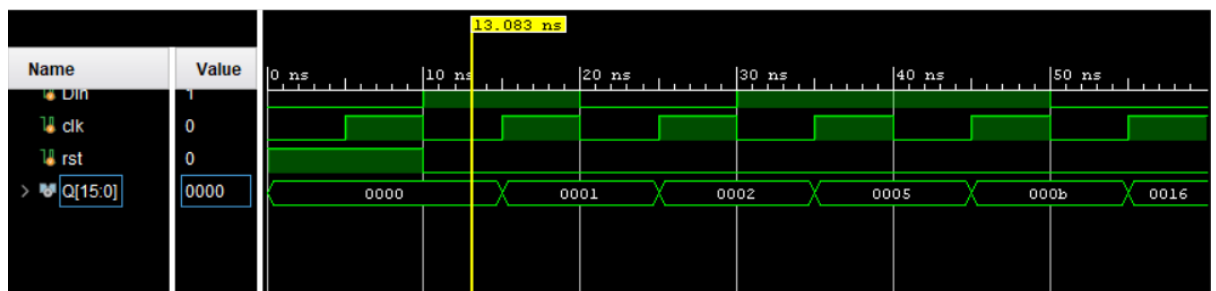
Testbench:

```

23 module shift_register_16_tb;
24
25     reg Din, clk, rst;
26     wire [15:0] Q;
27
28     shift_register_16 uut (Q, Din, clk, rst);
29
30     // Clock
31     always #5 clk = ~clk;
32
33     initial begin
34         $monitor("t=%0t Din=%b Q=%b", $time, Din, Q);
35
36         clk = 0; rst = 1; Din = 0; #10;
37         rst = 0;
38
39         // Apply input bits
40         Din = 1; #10;
41         Din = 0; #10;
42         Din = 1; #10;
43         Din = 1; #10;
44         Din = 0; #10;
45
46         $finish;
47     end
48
49 endmodule

```

Waveform for Shift register:



Test Cases For d_ff:

reset = 1'b1; in_d = 1'b1;

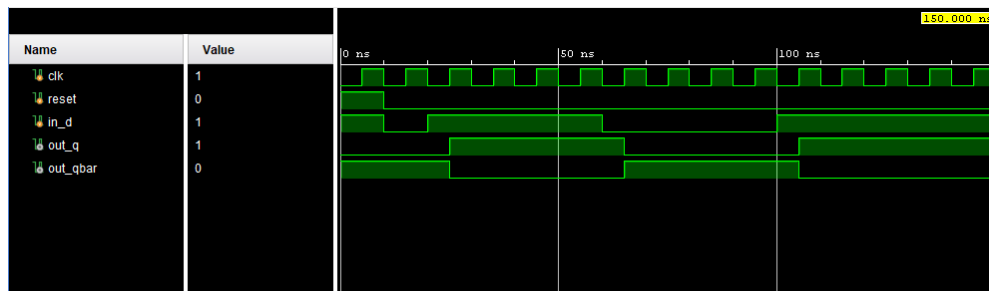
#10 reset = 1'b0; in_d = 1'b0;

#10 in_d = 1'b1;

#40 in_d = 1'b0;

#40 in_d = 1'b1;

Waveform for d_ff:



References:

- S. Ahmad, S. G. Khawaja, N. Amjad and M. Usman, "A Novel Multiplier-Less LMS Adaptive Filter Design Based on Offset Binary Coded Distributed Arithmetic
- Adaptive FIR Filter based on Distributed Arithmetic and LMS Algorithm for Low-Area and Low-Power
K. Jebin Roy M.E. VLSI Design Anand Institute of Higher Technology Affiliated to Anna University, Chennai – 603103, India
R. Ramya M.Tech. VLSI Design, Sathyabama University, Accredited by NAAC, Chennai – 600119, India
- FPGA – Based Adaptive FIR Filter Using LMS Algorithm
Heba H. Ali Department of Electronic Engineering College of Engineering University of Diyala Diyala, Isam S. Hameed Department of Electronic Engineering College of Engineering University of Diyala Diyala, Raghad A. Kareem Department of Electronic Engineering, College of Engineering University of Diyala Diyala, Salah H. Alkurwy Department of Electronic Engineering College of Engineering University of Diyala Diyala
- Low Power and Area Efficient FIR Filter Using Adaptive LMS Algorithm M. Dasharatha, B. Rajendra Naik and N.S.S. Reddy

A.2 Tools Used

Xilinx Vivado 2018.2

Matlab